

MERO KOTHA

The Complete Developer's Guide

Every Line of Code, Syntax, and Command Explained

A comprehensive reference manual detailing the MERN-stack architecture, database models, file upload parameters, CSS design systems, and payment structures of the Mero Kotha Birtamode rental portal.

TARGET AUDIENCE: Developer Community & Public Launch Reference

Author: Antigravity AI (Pair Programming)

Locality: Birtamode, Jhapa, Nepal

Developer eSewa Support ID: 9815810188

Date of Compilation: June 12, 2026

Technical Architecture & Database Design

Mero Kotha is developed using the MERN stack, separating concerns between a RESTful API (backend) and a single-page application (frontend). The project directory contains two isolated environments: backend and frontend, each running their own dependencies.

MongoDB Database Schema Design

The database layer utilizes Mongoose (MongoDB ODM) to define schemas and validate incoming data before insertion. Mero Kotha contains two main collections: Users (representing Property Owners) and Listings (representing rooms/flats).

1. User Model Schema (models/User.js)

The User schema stores registration data for property owners. Before saving the record to the database, a Mongoose pre-save hook intercepts the password field, generates a salt, and hashes the password using bcryptjs to ensure encryption.

- name: String (Required) - Stores the owner's full name.
- email: String (Required, Unique, Lowercase) - Used as the login identifier. Validated with a regex pattern.
- phone: String (Required) - Default contact number for property listings.
- password: String (Required, Minimum 6 characters) - Stored as a hashed string. The field is configured with `select: false` so it is not returned in queries by default.
- timestamps: Automatically adds `createdAt` and `updatedAt` fields.

2. Listing Model Schema (models/Listing.js)

The Listing schema stores room and house advertisements. It contains a reference link pointing back to the User who created it.

- owner: Mongoose Schema.Types.ObjectId (Ref: User, Required) - Creates a relational boundary link to the owner profile.
 - title: String (Required, Max 100 characters) - Clean text summarizing the space.
 - description: String (Required) - Detailed descriptions of amenities and rules.
 - ward: Number (Required, Range 1 to 10) - Validated to ensure listings belong strictly to Birtamode Municipality Wards 1 through 10.
 - location: String (Required) - Landmark or street-level address.
 - price: Number (Required) - Monthly rent price in Nepalese Rupees (NPR).
 - images: Array of Strings (Max 2 images) - Stores the static file paths of uploaded property pictures.
 - contactName: String (Required) - Display name for public listing view.
 - contactPhone: String (Required) - Public telephone number to facilitate direct calls.
 - timestamps: Tracks date of creation and modification.
-

Backend API Deep-Dive

The backend API handles authentication, listing queries, property posts, file uploads, and resource deletion.

Express Server Architecture (server.js)

The server entry point initializes express, enables CORS (Cross-Origin Resource Sharing), parses requests, and exposes the public folders.

- **CORS Middleware:** Enabled using `cors()` to allow the frontend client (running on port 5173) to communicate with the API server (running on port 5000) without browser blockages.
- **Static Route:** `app.use('/uploads', express.static(path.join(__dirname, 'uploads')))` - Enables public HTTP retrieval of uploaded room photos.
- **Error Handling Middleware:** A global try-catch boundary at the bottom of the middleware chain catches any unhandled runtime exceptions and returns a clean JSON error structure.

Authentication Routes & Middleware (routes/auth.js)

- **POST /api/auth/register:** Receives owner signup details, checks for duplicate emails, creates the record, and signs a JWT (JSON Web Token) with the user ID.
- **POST /api/auth/login:** Locates user email, compares password hashes, and generates a JWT token valid for 30 days.
- **GET /api/auth/profile (Protected):** Decodes JWT tokens to return currently logged-in user profile details (excluding the hashed password).
- **JWT Protection (middleware/auth.js):** A middleware function that intercepts authorization headers starting with "Bearer ", extracts the token, verifies it against the secret key using `jwt.verify`, and attaches the user model to the `req.user` object.

Listings and File Upload Router (routes/listings.js)

This module handles CRUD operations for rooms. It integrates Multer for disk storage and configures rules:

- **Multer Storage:** Saves files inside the "uploads/" directory with a unique timestamp suffix to prevent file name collisions.
- **Type Filter:** Limits file extensions strictly to jpeg, jpg, png, and webp.
- **Limit constraints:** Restricts file sizes to 5MB and files count to exactly/up to 2 per post.
- **POST /api/listings (Protected):** Accepts FormData, processes text attributes and uploaded file paths, and logs the document. If creation fails, it automatically unlinks (deletes) the uploaded files from disk.
- **GET /api/listings (Public):** Returns listings. Features query filtering for:
 - **ward:** Matches Birtamode Wards 1 to 10.
 - **minPrice / maxPrice:** Queries using Mongoose operators (`$gte` and `$lte`).
 - **search:** Performs regex search (`$regex` with case-insensitive option "i") across title, location, and description.
- **DELETE /api/listings/:id (Protected):** Checks if the requester is the owner of the listing. Deletes associated files from the disk and removes the record from the database.

Frontend Component & State System

The frontend is built as a single-page React app via Vite, compiled with Vanilla CSS styling for high-performance visual presentation.

Global Context State (src/context/AuthContext.jsx)

Provides centralized state management for authentication. It loads the active token from localStorage on boot, queries the profile route, and stores the user state. It exposes helper methods:

- login(email, password): Performs HTTP call, stores JWT token to localStorage, and updates state.
- register(name, email, phone, password): Submits register endpoint, logs in user automatically.
- logout(): Flushes localStorage and user state, redirecting them home.

Styling Framework & CSS Variables (src/index.css)

Styling relies entirely on native CSS variables (tokens) to manage theme constants:

- --primary: Crimson red (#A51D24) representing Nepal's national color.
- --accent: Gold (#D4AF37) for highlights and badge details.
- --esewa: Green (#60BB46) representing eSewa.
- Fonts: Headings use "Plus Jakarta Sans" for modern premium display, and body text uses "Inter" for clean readability.
- Modals & overlays: Uses backdrop-filter: blur(4px) for frosted-glass aesthetic overlays.

Key Page Modules (src/pages/)

1. Browse & Filter Page (Home.jsx)

Features a sticky search and filter panel allowing visitors to filter by keyword, select any Birtamode Ward (1-10), and define budget limits. An asynchronous effect triggers API fetches on filter adjustments.

2. Property Display Details (ListingDetail.jsx)

Implements an interactive double-image display. Clicking on thumbnail previews swaps the main image view. Displays rent rates, ward designations, calendar post dates, and a call-to-action button linked directly to the owner's phone via the tel protocol.

3. Listing Creation Form (CreateListing.jsx)

Utilizes standard HTML5 Form Validation and a custom image file selector. It checks image sizes (5MB max) and limits array lengths to 2. It submits files using JavaScript FormData instead of raw JSON to support multi-part binary transfer.

eSewa Payment Integration

Mero Kotha features a developer support modal. If you wish to implement a pay-to-publish paywall for posting listings, the process uses the eSewa gateway.

eSewa Integration Protocol

- 1. Register: Get a Merchant ID and Secret Key from eSewa (or use "EPAYTEST" for sandbox testing).
- 2. Draft Listing: When an owner clicks "Publish", the backend saves the listing with status: "pending_payment".
- 3. Parameter Signature: The backend generates a secure SHA-256 HMAC signature of the transaction details (total amount, product ID, merchant code) using the merchant key.
- 4. Redirect: React submits a form POST to eSewa's portal (<https://uat.esewa.com.np/epay/main>) with signature and transaction params.
- 5. Redirect Callback: After payment, eSewa redirects to your success endpoint. The backend calls eSewa server-to-server to verify the transaction reference, marks status: "active", and redirects the user back to the React success dashboard.

Commands Reference & Setup Log

Use these commands to manage your local environments:

Backend Commands

- Initialize project: `npm init -y`
- Install packages: `npm install express mongoose dotenv cors jsonwebtoken bcryptjs multer`
- Start development server (nodemon): `npm run dev`
- Start standard server: `npm start`

Frontend Commands

- Create Vite project: `npx create-vite@5 frontend --template react`
 - Install packages: `npm install react-router-dom lucide-react`
 - Launch local dev server: `npm run dev`
 - Build production bundle: `npm run build`
-

